

Plant Biomass Prediction - MLOps Praktikum 1 - Gruppe 2

This project implements a machine learning pipeline for plant biomass using top-down images of plants. The machine learning pipeline includes Exploratory Data Analysis, a model training workflow using PyTorch, git and automated logging.

1. Dataset Description & Statistics

Samples: 4294 images Target Variable: fresh_weight_total Features: Temporal data, sensor data Input Data: colored images located in "mlops_biomass_data"/"images_med_res"

2. Exploratory Data Analysis

2.1 Target Distribution By looking at the graph, we can see that the biomass labels are right-skewed. This means that we have a high count of lightweight plants, which indicates that they are in an early growing phase.

2.2 Correlation Heatmap Humidity and temperature sensors are redundant. They do not give us new information. The total fresh weight correlates positively with the weight of the shoot. It also correlates positively with the age and the total leaves. This is logical and normal.

2.3 Biomass vs Age With continuing age the total biomass increases. This is a sign for growth. It also looks like the biomass got measured once per week, because there aren't dots at every day.

2.4 Image Pixel Analysis The graph spikes at 90 on the pixel distribution scale. This is pretty dark. The model sees a lot of dirt, which is darker than the green plants.

2.5 Sample images We can see that plants, which have bigger leaves also have a bigger mass. This confirms our other observations.

3. Data Quality Issues

Missing Labels: `df.dropna()` found empty rows, so we had to implement that invalid labeled samples are removed automatically.

Target Scale: Initial loss was "NaN" due to high mean squared error values. Labels were scaled from 0 to 1 by dividing each value by the highest value.

4. Model Architecture

We implemented a ResNet-18 architecture, which is a deep residual network. By using ResNet-18, we took advantage of the model pre-trained on the ImageNet dataset, which made it easier to detect visual parts of the plants (f.e. texture or edges). We modified it to predict the continuous weight value instead of 1000 classes (`model.fc = nn.Linear(model.fc.in_features, 1)`). Therefore we shifted the task from classification to regression.

5. Training Procedure

The training was conducted on a CPU/GPU-agnostic pipeline with the following settings:

Optimizer: Adam

Loss Function: Mean Squared Error (MSE)

Learning Rate: 0.0001

Batch Size: 16

Epochs: 3

Data Split: 80% Training / 20% Validation (Seed: 42).

6. Results

6.1 Training Curves The loss decreases over each epoch. The Validation Loss follows the Training Loss. The Model is not overfitting.

6.2 Metrics Final_Train_Loss: 0.0044 Final_Val_Loss: 0.0072
Max_Weight_Scale: 2.113

Labels were scaled by the maximum weight found in the dataset.

7. Challenges & Improvements

7.1 Challenges Exploding Gradients: Initial training failed with “NaN” loss because the biomass weights were too large for an unscaled regression head.

7.2 Improvements We can implement RandomRotate90 and ColorJitter using Torchvision transforms. This forces the model to recognize plants from different angles and with different lighting conditions.

By using Vision Transformer (ViT) the model could learn that distant leaves are part of the same plant.

8. Reproduction Instructions

1. Clone the Repository

```
git clone https://gitlab.nt.fh-koeln.de/gitlab/mlops/praktikum/mlops_2/MLOps_P1_2.git  
cd MLOps_P1_2
```

2. Setup Environment

```
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

3. Run the EDA

```
python eda.py
```

4. Run the Model Training

```
python train_model.py -epochs 3 -lr 0.0001
```